

26F Stat TA Session W9

0. Preparation for TA Session

- Download (and import) the sample code and attached datasets: `babies` & `restaurants` [click here](#)
 - the `babies` dataset includes the babies born in 1960-1967 in the San Francisco East Bay area
 - the `restaurant` dataset is credited to the TA of last year!!!

1. Logical Expressions

- This week, we will introduce you to the world of data wrangling (limbo). But before we run into the trivial work, let's first take a look at the useful logical expressions

Comparison Operators `>` , `<` , `=` , `!`

- You should be familiar with these operators (except for the last one `!` (negative))

```
# > (Greater than)
print(5 > 4) # TRUE

# >= (Greater or equal)
print(3 >= 4) # FALSE

# < (Smaller than)
print(5 < 4) # FALSE

# <= (Smaller or equal)
print(3 <= 4) # TRUE

# == (Equal to)
print(4 == 4) # TRUE

# != (Not equal to)
print("H" != "I") # TRUE
```

Logical Combination `&` , `&&` , `|` , `||`

- You can also combine multiple statements to form a more complex logical statement

AND `&` (element wise), `&&` (first element only)

```
x <- c(3, 5, 8)
x > 4 & x < 7 # FALSE TRUE FALSE
x > 4 && x < 7 # Error: 'length(x) = 3 > 1' in coercion to 'logical(1)'
```

OR `|` (element wise), `||` (first element only)

```
x <- c(3, 5, 8)
x > 4 | x < 7 # TRUE TRUE TRUE
x > 4 || x < 7 # Error: 'length(x) = 3 > 1' in coercion to 'logical(1)'
```

Q. How to choose between element wise (`&`, `|`) and first element only (`&&`, `||`)?



Ans. Choose first element only one if

- somehow you only want to compare the first element in the vectors
- it's inside an `if` statement or a `for` loop (there should be only a single logical condition for control flows)

```
# example
if (TRUE || FALSE){
  print("TRUE!")
}
```

- `&` and `|` are also useful filters

```
df <- data.frame(age = c(16, 22, 30, 40),
                  score = c(80, 90, 55, 60))
df[df$age > 18 & df$score > 70, ] # which rows to keep

# output
  age score
2  22    90
```

`%in%`

- Recall that we have encountered this operator in W6 exercises, it is used to check if an object is inside a vector or a list

```
# example
fruit <- c("apple", "banana", "pear")
basket <- c("banana", "orange", "pear", "kiwi")
fruit %in% basket # FALSE TRUE TRUE

# example
3 %in% list(1, 2, 3, 4, 5) # TRUE
3 %in% list(1:5, 6:10) # FALSE
1:5 %in% list(1:5, 6:10) # FALSE FALSE FALSE FALSE FALSE
list(1:5) %in% list(1:5, 6:10) # TRUE
```

Use `()` for Clearer Expressions

- Coding Style matters! When the logical expression is too complicated or too long, add `()` to improve the readability! (Also save your time from debug)
- `()` can also help you understand the hierarchy (order) of operators (just like `()` in math calculations)

```
# example
x <- 7

# comparisons (>, <, ==) are evaluated before &
# & is evaluated before |
x > 4 & x < 10 | x == 2 & x > 8 # TRUE
# this is equivalent to
(x > 4 & x < 10) | (x == 2 & x > 8) # TRUE

# however, if you meant to calculate from left to right
((x > 4 & x < 10) | x == 2) & x > 8 # FALSE
```

2. Data Wrangling with `dplyr`

- Just as plotting with `ggplot2`, we (hopefully) have a useful and powerful tool for data wrangling, named `dplyr`

`dplyr` and `tidyverse`

- `dplyr` is grammar of data manipulation in `R`
 - Core functions: `filter()`, `select()`, `mutate()`, `arrange()`, `summarize()`, `group_by()`
 - Another useful tool: `%>%`
- `tidyverse` is a collection of packages including `ggplot2`, `dplyr`, `tidyr`, `readr`, etc.
 - You can also load `tidyverse` in `RStudio` without further specifying which exact packages you are using (e.g., `dplyr`)

Pipeline `%>%`

- The pipeline operator `%>%` in `dplyr` is used to chain multiple operators on data together

```
# example
num <- -3:3
print(num)

num %>%
  cumsum() %>%
  print()

# output
```

```
[1] -3 -2 -1  0  1  2  3
[1] -3 -5 -6 -6 -5 -3  0
```

What's in the Dataset?

- Now let's take a look at the `babies` dataset
 - `case`: id number
 - `bwt`: birthweight
 - `age`: mother's age in years
 - `height` / `weight`: mother's height/ weight in inches/ pounds

	case	bwt	gestation	parity	age	height	weight	smoke
1	1	120	284	0	27	62	100	0
2	2	113	282	0	33	64	135	0
3	3	128	279	0	28	64	115	1
4	4	123	NA	0	36	69	190	0
5	5	108	282	0	23	67	125	1

`filter()`

- Suppose we want to filter the mothers with height 62-64 inches (~ 157.5-162.5 cm), we use `%>%` and `filter()`

```
babies_height_62_64 <- babies %>%
  filter(height %in% 62:64) # height is recorded in integers

print(babies_height_62_64)

# output (only the first 5 rows shown)
  case bwt gestation parity age height weight smoke
1     1 120      284     0  27     62    100     0
2     2 113      282     0  33     64    135     0
3     3 128      279     0  28     64    115     1
4     6 136      286     0  25     62     93     0
5     7 138      244     0  33     62    178     0
```

- Another example can be like looking for smoking mom and babies with birth weight > 120 (there are 171 such examples...)

```
# example
babies_bwt_gt120_smoke <- babies %>%
  filter(bwt > 120 & smoke == 1)

print(babies_bwt_gt120_smoke)

# output (only the first 5 rows shown)
  case bwt gestation parity age height weight smoke
```

1	3	128	279	0	28	64	115	1
2	10	143	299	0	30	66	136	1
3	12	144	282	0	32	64	124	1
4	13	141	279	0	23	63	128	1
5	38	134	288	0	23	63	92	1

- You may have noticed that there are `NA` values (missing values) in the dataset
- By default, `filter()` does not keep rows with `NA`

`select()`

- There are too many columns (variables) in the dataset. Suppose now we only want to focus on columns `bwt` and `age`, we can use `select()` to select and keep these columns

```
babies_age_bwt <- babies %>%
  select(bwt, age)

print(babies_age_bwt)

# output (only the first 5 rows shown)
  bwt age
1 120 27
2 113 33
3 128 28
4 123 36
5 108 23
```

- We may also delete certain columns

```
# example
babies_wo_case_bwt <- babies %>%
  select(-c(case, bwt)) # delete columns case and bwt

print(babies_wo_case_bwt)

# output (only the first 5 rows shown)
  gestation parity age height weight smoke
1      284      0  27    62    100      0
2      282      0  33    64    135      0
3      279      0  28    64    115      1
4       NA      0  36    69    190      0
5      282      0  23    67    125      1
```

Create New Columns by Existing Variables `mutate()`

- In most cases, the variable of our interests may not be recorded in the original dataset (e.g., the BMI of moms). Then, we have to compute the numbers and create a column ourselves

- The BMI is left as exercise (hint: be aware of the unit used!). In the example, we show how to create a column for height in cm

```
babies_cm <- babies %>%
  mutate(height_cm = height*2.54)
```

	case	bwt	gestation	parity	age	height	weight	smoke	height_cm
1	1	120	284	0	27	62	100	0	157.48
2	2	113	282	0	33	64	135	0	162.56
3	3	128	279	0	28	64	115	1	162.56
4	4	123	NA	0	36	69	190	0	175.26
5	5	108	282	0	23	67	125	1	170.18

- `mutate()` is also very powerful for condition grouping!
 - If the grouping condition is binary, we can use together with `ifelse()`
 - If groups ≥ 3 , use together with `case_when()`

```
# example
babies_age_gt30 <- babies %>%
  mutate(age_30 = ifelse(age > 30, "age > 30", "age <= 30"))
```

	case	bwt	gestation	parity	age	height	weight	smoke	age_30
1	1	120	284	0	27	62	100	0	age <= 30
2	2	113	282	0	33	64	135	0	age > 30
3	3	128	279	0	28	64	115	1	age <= 30
4	4	123	NA	0	36	69	190	0	age > 30
5	5	108	282	0	23	67	125	1	age <= 30

```
# example
babies_ht_group <- babies %>%
  mutate(ht_group = case_when(
    is.na(height) ~ NA, # condition ~ value
    height <= 63 ~ "ht <= 63",
    height > 63 & height < 67 ~ "63 < ht < 67",
    TRUE ~ "ht >= 67" # as the default (catch-all)
  ))
```

	case	bwt	gestation	parity	age	height	weight	smoke	ht_group
1	1	120	284	0	27	62	100	0	ht <= 63
2	2	113	282	0	33	64	135	0	63 < ht < 67
3	3	128	279	0	28	64	115	1	63 < ht < 67
4	4	123	NA	0	36	69	190	0	ht >= 67
5	5	108	282	0	23	67	125	1	ht >= 67

Sort the Data `arrange()`

- Suppose we want the rows to be in descending order according to the mother's height
 - By default, the order is ascending

```
# example
babies_ht_desc <- babies %>%
  arrange(desc(height)) # desc = descending; default = ascending

print(babies_ht_desc)
```

```
# output
  case bwt gestation parity age height weight smoke
1  176 122      278      0  31    72    155      1
2   86 129      274      0  29    71     NA      1
3  148 160      300      0  29    71    175      1
4  288 148      279      0  27    71    189      0
5  294 121      276      1  23    71    152      1
```

```
# example
babies_ht_asc <- babies %>%
  arrange(height)

print(babies_ht_asc)
```

```
# output
  case bwt gestation parity age height weight smoke
1  435 146      263      0  39    53    110      1
2 1209 141      281      0  29    54    156      1
3   21 105      270      0  22    56     93      0
4  668  99      249      1  31    57     98      1
5   27 146      280      0  26    58    106      0
```

Caveats

- These `dplyr` functions do not change the original dataset (unless you assign the output back to the variable itself)
- Always use pipeline to improve readability and keep your code maintainable

```
# example

# Without %>%
arrange(select(filter(babies, height >= 60),
                  bwt,
                  height),
        desc(bwt)
)

# With %>%
```

```
babies %>%  
  filter(height >= 60) %>%  
  select(bwt, height) %>%  
  arrange(desc(bwt))
```

Exercises

W9 Exercises